

Hermes Agent Project Blueprint — Design Spec

Context

The Hermes Agent project needs a comprehensive onboarding blueprint for new developers joining the team. The project already has `AGENTS.md` (reference guide for AI coding assistants, how-to-add-things) and `CONTRIBUTING.md` (contributor workflow, PR process, code style). The blueprint fills a distinct gap: the "understand the whole system" document — architectural context, design rationale, tribal knowledge, and a structured onboarding path.

Target Audience

New developers joining the Hermes team. They need to understand the full system architecture, learn why decisions were made, avoid known pitfalls, and have a concrete path to productivity.

Format

Single `BLUEPRINT.md` file at the project root. Strong table of contents for navigation. Complements (does not duplicate) `AGENTS.md` and `CONTRIBUTING.md`.

Sections

1. Welcome & Quick Start

- One-paragraph project identity (self-improving multi-platform AI agent, learning loop, run-anywhere, 15 platforms)
- Dev setup summary (references `CONTRIBUTING.md` for full details)
- Three runtime modes: CLI, Gateway, Batch — with one-liner examples
- Quick reference table of `~/hermes/` directory layout

2. Architecture Deep-Dive

- System-level ASCII diagram: three runtime modes → AI Agent → LLM providers, tool registry, session DB, memory

- Agent loop walkthrough: `run_conversation()` step-by-step (prompt building → API call → tool dispatch → result injection → context check → loop/return)
- Message format: OpenAI-compatible schema, reasoning content storage, tool result structure
- Provider abstraction: OpenAI/Anthropic/OpenRouter/custom endpoints, `anthropic_adapter.py`, credential resolution via `hermes_cli/auth.py`, credential pooling (`credential_pool.py`)
- Prompt assembly pipeline: `prompt_builder.py` order (identity → tool guidance → memory guidance → skills index → project context), prompt caching implications
- Context management: token estimation (`model_metadata.py`), compression trigger and 5-phase algorithm (`context_compressor.py`), what gets summarized vs preserved
- Session persistence: SQLite WAL mode with FTS5 (`hermes_state.py`), session/message schema, JSON logs as secondary format
- File dependency chain: `tools/registry.py` → `tools/*.py` → `model_tools.py` → `run_agent.py` / `cli.py` / `gateway`

3. Component Guides

3a. Tool System

- Self-registration pattern: `registry.register()` at import time, `_discover_tools()` triggers imports
- Anatomy of a tool: schema, handler, `check_fn`, toolset assignment, `requires_env`
- `ToolRegistry` dispatch flow: `handle_function_call()` → registry lookup → handler → JSON string result
- Async bridging: `_run_async()` with per-context event loop strategy
- Toolset grouping (`toolsets.py`): core toolsets, platform presets, recursive resolution via `resolve_toolset()`
- Approval system (`approval.py`): dangerous command detection, per-session allow-listing
- Agent-level tools (todo, memory): intercepted before `handle_function_call()`
- Terminal backends (`tools/environments/`): local, Docker, SSH, Modal, Daytona, Singularity — `BaseEnvironment` ABC

3b. CLI & TUI

- `HermesCLI` lifecycle: config loading → skin init → `prompt_toolkit` session → agent loop
- Slash command registry: `CommandDef` in `commands.py` — single source of truth for CLI, gateway, Telegram menus, Slack, autocomplete
- Adding a slash command: 3-step process (`CommandDef` → CLI handler → optional gateway handler)
- Skin engine (`skin_engine.py`): data-driven theming, built-in vs user YAML skins, what skins customize
- Callbacks (`callbacks.py`): clarify, sudo, approval flows
- Profile system: multi-instance isolation via `HERMES_HOME` env var override, `get_hermes_home()` everywhere

3c. Gateway & Messaging Platforms

- `GatewayRunner` architecture: platform lifecycle, message routing, agent caching per session
- Platform adapter ABC (`base.py`): `connect()` , `disconnect()` , `send()` , `get_chat_info()` required; `edit_message()` , `send_typing()` , `send_image()` , `send_voice()` optional
- Full platform list: Telegram, Discord, Slack, WhatsApp, Signal, Email, Matrix, Mattermost, Home Assistant, DingTalk, Feishu, WeCom, SMS, Webhook, API Server
- Message flow: platform event → `MessageEvent` normalization → `GatewayRunner.handle_message()` → `AI Agent` → response → adapter → platform
- Session management (`session.py`): `SessionStore` , `SessionContext` , `SessionSource` , `build_session_key()` , PII redaction with hashing
- Session context prompt: dynamic injection telling agent where messages come from and what platforms are connected
- Media caching: platform images/audio/docs cached to `~/.hermes/cache/` with age-based cleanup
- Token locks: prevent two profiles from using the same bot credential
- Adding a new platform: implement `BasePlatformAdapter` , register in gateway config

3d. Skills System

- Skills vs tools decision framework (instructions + scripts vs Python API integration)
- `SKILL.md` format: frontmatter schema (name, description, version, platforms, `required_environment_variables`, metadata)
- Skill loading: `prompt_builder.py` builds skills index with two-layer cache (in-process LRU + disk snapshot)
- Conditional activation: `fallback_for_toolsets` , `requires_toolsets` , `fallback_for_tools` , `requires_tools`
- Platform filtering: `platforms` field restricts to macOS/Linux/Windows
- Skill categories: bundled (`skills/`), optional (`optional-skills/`), hub (community)
- Agent-created skills: the self-improvement loop
- Skills Hub: discovery, installation, publishing via `skills_hub.py`
- Skill setup metadata: `required_environment_variables` for secure load-time credential collection
- Skills guard (`skills_guard.py`): security scanning for hub-installed skills
- Skill guidelines: no external deps, progressive disclosure, include helper scripts, test with `hermes --toolsets skills -q`

3e. RL Training & Trajectory System

- Trajectory saving: `agent/trajectory.py` — how conversations become training data
- Batch runner (`batch_runner.py`): `BatchRunner` class, multiprocessing pool, checkpointing, `--resume` , tool statistics aggregation, reasoning filtering

- Trajectory compression (`trajectory_compressor.py`): `CompressionConfig` , 5-phase strategy (protect head/tail, summarize middle via LLM), JSONL output with metrics
- RL environments (`environments/`): `HermesAgentBaseEnv` extending `Atropos BaseEnv` , `HermesAgentEnvConfig` , subclass interface (`setup()` , `get_next_item()` , `format_prompt()` , `compute_reward()` , `evaluate()`)
- Existing environments: web research (FRAMES), agentic OPD, terminal bench, YC bench
- `rl_cli.py` : RL training runner, `RL_MAX_ITERATIONS=200` , tinker-atropos submodule checks
- `mini_swe_runner.py` : lightweight SWE task executor with pluggable backends (local/Docker/Modal), JSONL trajectory output

3f. Supporting Systems

- Cron scheduler (`cron/`): natural language job creation, `croniter` parsing, scheduled execution
- MCP server (`mcp_serve.py`): stdio-based, 10-tool surface (`conversations_list`, `conversation_get`, `messages_read`, `attachments_fetch`, `events_poll`, `events_wait`, `messages_send`, `channels_list`, `permissions_list_open`, `permissions_respond`), connects SessionDB + routing metadata
- ACP adapter (`acp_adapter/`): VS Code / Zed / JetBrains Copilot integration
- Honcho integration (`honcho_integration/`): `HonchoSessionManager` , persistent cross-session user modeling, configurable write frequency, runs alongside SQLite + file memory

4. Design Decisions & Rationale

- **OpenAI message format as internal standard** — Provider-agnostic default, Anthropic adapter translates, avoids lock-in
- **Synchronous agent loop** — Simpler to reason about and debug, async only at gateway level where needed
- **Self-registering tools at import time** — No central manifest, adding a tool = file + one import line, registry always in sync
- **Skills over tools for most capabilities** — Skills are instructions (cheap, no code changes, community-contributed), tools are Python (heavyweight, require releases)
- **SQLite with FTS5** — Zero-dependency, single-file, fast full-text search, no external DB to configure
- **Prompt caching as a first-class constraint** — Cost difference is dramatic, drives the no-mid-conversation-changes rule
- **HERMES_HOME env var for path isolation** — One override before import gives every module the right path for profiles
- **Ephemeral injection for system prompts** — Not persisted to DB/logs, keeps sessions clean, avoids leaking internals
- **Pluggable terminal backends** — Same agent logic runs locally, in Docker, over SSH, or serverless
- **WAL mode with random jitter retry** — Breaks convoy effects in multi-process gateway, better than deterministic backoff

- **Credential pooling with exhaustion tracking** — Same-provider failover, multiple auth sources, configurable selection strategies

5. Gotchas & Tribal Knowledge

Note: AGENTS.md "Known Pitfalls" section covers specific coding rules (hardcoded paths, ANSI escapes, menu libraries, etc.). This section focuses on the **architectural reasoning** behind those rules and adds cross-cutting concerns that don't fit in a single-file reference.

Profile isolation is load-bearing architecture: The `HERMES_HOME` env var is set *before any module imports* in `hermes_cli/main.py: _apply_profile_override()`. Every module that calls `get_hermes_home()` at import time gets the right path automatically. This is why hardcoding `~/.hermes` breaks profiles — it bypasses the entire isolation mechanism. This single pattern broke 5 times in PR #3575 because contributors didn't understand the load order.

Prompt caching economics drive architecture: The rule against mid-conversation changes (no toolset swaps, no memory reloads, no system prompt rebuilds) isn't arbitrary caution — it's driven by cost. Anthropic prompt caching gives dramatic savings only when the prefix is stable. Breaking the cache mid-conversation can 10x the cost of a session. Context compression is the *only* allowed exception because it's a last resort before hitting context limits.

The tool system has two dispatch layers: Agent-level tools (todo, memory, session_search) are intercepted in `run_agent.py` *before* `handle_function_call()`. Everything else goes through `tools/registry.py`. If you add a tool and it's not being called, check whether it's being intercepted at the wrong layer.

Async bridging is subtle and context-dependent: `model_tools.py: _run_async()` uses three different strategies depending on the calling context (gateway async loop, worker thread, CLI main thread). If you see "Event loop is closed" errors, the bridging strategy is probably wrong for your context.

Cross-platform is not optional: Hermes runs on Windows, macOS, and Linux in production. `termios` / `fcntl` are Unix-only, `.env` encoding varies, process management differs. Every PR touching OS-level code needs Windows consideration. See AGENTS.md "Known Pitfalls" and CONTRIBUTING.md "Cross-Platform Compatibility" for the specific rules.

Test isolation is enforced by fixture: `_isolate_hermes_home` autouse fixture in `tests/conftest.py` redirects all state to tmp dirs. Profile tests additionally need `Path.home()` mocked because `_get_profiles_root()` is HOME-anchored, not HERMES_HOME-anchored. If your test is reading/writing real `~/.hermes/`, the fixture isn't working — investigate, don't bypass.

Config migration is versioned: Adding to `DEFAULT_CONFIG` requires bumping `_config_version` in `hermes_cli/config.py` (check the current value there). This triggers automatic migration for existing users. Forgetting this means existing installs silently miss the new option.

Context files have safety limits: Project context files (`.hermes.md`, `AGENTS.md`, etc.) are truncated at 20K chars (70% head + 20% tail) and scanned for prompt injection patterns by `_scan_context_content()`. If a legitimate context file is being flagged or truncated, understand these limits before changing them.

6. Your First Week

Day 1-2: Get Oriented

- Set up dev environment, run `hermes doctor`, run full test suite (`pytest tests/ -v`)
- Read blueprint sections 1-3 (overview, architecture, component guides intro)
- Start a CLI session, have a conversation, inspect the session in `~/.hermes/state.db`
- Trace a single message: user input → `cli.py` → `run_agent.py:run_conversation()` → `model_tools.py:handle_function_call()` → tool execution → response

Day 3: Tools & Skills

- Read `tools/file_tools.py` (straightforward tool example)
- Create a toy tool: register it, add to a toolset, verify it appears in the agent's tool list
- Read a skill (`skills/research/` is a good starting point)
- Load a skill in a session, observe how it appears in the system prompt via `prompt_builder.py`

Day 4: Gateway

- Run the gateway locally with Telegram (create a bot via BotFather)
- Trace a gateway message: Telegram adapter → `GatewayRunner.handle_message()` → `AIAgent` → response
- Read `gateway/platforms/base.py` to understand the adapter contract
- Read `gateway/session.py` to understand session key generation and PII redaction

Day 5: RL & Trajectories

- Run a CLI session with `--save-trajectories`, inspect the output
- Read `trajectory_compressor.py` — understand the 5-phase compression strategy
- Browse `environments/hermes_base_env.py` — understand the subclass interface
- Run `batch_runner.py` with a small dataset to see parallel generation

Day 6-7: Go Deeper

- Pick a recent bug fix from git log, read the PR, understand what broke and why
- Pick an area of interest, make a small improvement (fix a typo, add a test, improve an error message)
- Submit your first PR following CONTRIBUTING.md guidelines

Cross-References

- `AGENTS.md` — Developer reference for AI assistants, how-to guides for adding tools/commands/config
- `CONTRIBUTING.md` — Contributor workflow, PR process, code style, security considerations
- `cli-config.yaml.example` — Full configuration reference with all options
- `.env.example` — All supported environment variables